Title Page

Project Name: Decision Companion - AI Decision Making System
Prepared by: [My Team/Name]

Abstract:
Decision-making is a fundamental cognitive process that becomes increasingly challenging as the complexity and stakes of the choices rise. In an era saturated with information, individuals and organizations frequently suffer from decision fatigue and cognitive overload. The "Decision Companion" is an innovative, cross-platform mobile application meticulously designed to assist users in navigating complex decisions through a structured, mathematically sound, and AI-powered framework. Built with a responsive Flutter mobile frontend utilizing Provider state management, and a scalable, robust Django REST Framework backend, the system integrates advanced Artificial Intelligence—specifically leveraging Anthropics's Claude 3.5 Sonnet via the OpenRouter API—to provide intelligent, contextual insights. Users can track multiple factors (Pros/Cons), apply highly customizable mathematical weights based on personal priorities, and generate automated confidence scores utilizing deeply nested relational database structures (Options, Factors, and Factor Ratings). The architecture employs JSON Web Tokens (JWT) for secure, stateless authentication and utilizes a robust relational database (SQLite/PostgreSQL) to safely persist user data, customizable global decision templates, and post-decision journaling reflections. By systematically bridging the gap between flawed human intuition and data-driven computational analysis, Decision Companion transforms anxiety-inducing dilemmas into quantifiable, manageable, and actionable steps, fostering a cycle of continuous personal development.

---

1. Chapter One

1.1 Introduction

The modern world presents individuals, business leaders, and everyday consumers with an ever-expanding, overwhelming number of choices. These range from routine daily operational tasks to life-altering career pivots and major financial investments. As the sheer volume of variables and potential outcomes increases, the cognitive load required to effectively process and weigh these factors often leads to a phenomenon known as decision fatigue, or worse, decision paralysis. Technology—specifically the rapid advancements in Large Language Models (LLMs) and ubiquitous mobile computing—offers a highly compelling solution by providing structured computational methodologies to quantify abstract concepts. "Decision Companion" is introduced in this thesis as a digital facilitator. It is a comprehensive system that explicitly marries established decision matrix theories with modern Generative AI to dramatically streamline the choice-selection process.

1.2 Background

Extensive psychological studies indicate that the human brain relies heavily on heuristics—mental shortcuts—to make decisions under uncertainty. While these approaches are evolutionarily efficient, they

are notoriously susceptible to debilitating cognitive biases such as anchoring bias, confirmation bias, and emotional overriding. Traditionally, individuals intending to make rational choices have relied on simple physical lists (the classic "Pros and Cons" sheet) or rudimentary digital spreadsheets to counteract these systemic biases. However, these traditional tools inherently lack dynamic adaptability, analytical depth, and the ability to parse complex multi-variable structural relationships in real-time. This highlights a pressing, unfulfilled need within the consumer software ecosystem for a dynamic solution capable of parsing multi-dimensional choices and offering highly objective, algorithmic insights securely.

1.3 Motivation

The conception and subsequent development of Decision Companion stem directly from observing the widespread difficulty individuals consistently face when navigating high-stakes personal and professional decisions. During our preliminary market analysis, we recognized a significant gap: while basic note-taking, simplistic polling apps, and highly expensive enterprise-level decision software exist, there is a distinct lack of accessible, mobile-first consumer platforms that explicitly guide a user through a mathematically sound and AI-augmented workflow. By wrapping computationally complex relationships (Options -> Factors -> Weighted Ratings) and state-of-the-art AI analysis in an intuitive, consumer-friendly mobile UI, we aim to systematically alleviate decision anxiety and measurably improve long-term decision outcomes.

1.4 Problem Definition

Users frequently struggle to logically and objectively evaluate multiple competing options against varying, weighted criteria. When faced with a momentous choice (e.g., purchasing real estate, selecting a university degree program), an individual must balance subjective criteria (e.g., 'cultural fit') against objective constraints (e.g., 'budget limits'). The structural problem is the massive cognitive inability for a human to accurately and repeatedly compute aggregate scores across dozens of intermingled Pros and Cons without mathematical bias, track how variables change over extended evaluation periods, and execute a final choice without lingering post-decision cognitive dissonance.

1.5 Why Decision Companion?

Decision Companion provides distinct and unique value by moving computationally far beyond traditional binary listing methodologies. Its standout proprietary features include:

1. **The Dynamic Factor Weighting API:** Which mathematically scales specific factors against user-rated inputs.
2. **LLM-Based Data Synthesization:** Automatically generating prompts from localized database states and securely passing them to Claude 3.5 Sonnet to highly contextualize outcomes, actively identifying strategic blind spots.
3. **The Reflection Journal System:** A distinctly psychological module directly correlating long-term emotional satisfaction back to the exact initial mathematical variables chosen months prior.

1.6 Objectives

This project achieved the following strictly measurable engineering objectives:

- **Automate Objective Analytical Calculation:** Implement an intelligent relational scoring system using Django to dynamically parse foreign-keyed Options and Options Ratings.
- **Provide Advanced AI Insights:** Leverage the OpenRouter platform API to request structured JSON analysis detailing confidence scores (0.0-1.0 float), itemized option validations, and explicit risk enumerations.
- **Ensure Persistent, Secure Decision Tracking:** Develop a cloud-synced, encrypted backend infrastructure utilizing Custom Django Users to safely save, revisit, and seamlessly modify ongoing decisions.
- **Deliver a Seamless Mobile Architecture:** Construct a rigid, scalable Flutter architecture successfully implementing Dependency Injection (`ServiceLocator`), Provider State Management, and standard `http` packages for utterly robust API connectivity.

1.7 Expected Outcome

The final deployed ecosystem reliably delivers a fully functional, highly performant cross-platform mobile application securely supported by a RESTful Django backend API. End users are empowered to instantly define strict criteria with granular weights, request an asynchronous AI analytical report parsing their localized SQLite/Postgres rows, and securely track their psychological decision history longitudinally.

---

2. Chapter Two

2.1 Related Works

An analysis of existing tools in the consumer productivity market reveals a segmentation into two distinct groups: general-purpose note-taking applications (Evernote, Notion) and specialized business analysis tools. Generic consumer apps provide absolute freedom but offer absolutely zero computational support or structured API inference. Users must fundamentally guess their final outcome based on text blocks. Conversely, specialized business tools are overwhelmingly rigid in their data schemas and prohibitively expensive. Crucially, neither category natively leverages personalized, Generative AI guidance to digest relational arrays and return strongly typed advice directly inside the native mobile experience.

2.2 Comparative Studies

Compared to traditional manual analytical lists (where factors are often instinctively viewed with equal psychological weight), Decision Companion completely alters the mathematical paradigm by deploying a strict Relational Database Weighting Model mapped to floating-point calculations. Manual lists rely on the user visually estimating “heaviness”—a process prone to massive mathematical error. The proprietary

API algorithm normalizes `FactorRating` mappings, making the mathematical disparity between `DecisionOption` instances categorically undeniable. Moreover, Decision Companion injects qualitative, generative AI analysis that dynamically reads those exact relational arrays to mathematically defend the recommendation in human-readable JSON payloads.

Data Flow Comparative Diagram:  Data Flow Diagram

2.3 Scope of the Problem

The operational scope of the platform is strictly constrained to handling discrete, multi-variable choice scenarios across standard contexts. It is explicitly architected for singular end-users via JWT session authentication, thus gracefully scoping out highly complex synchronous multi-user enterprise WebSockets. The system's Artificial Intelligence footprint is carefully restricted; it relies entirely on the OpenRouter API via securely isolated, heavily sanitized string prompts containing strictly localized contextual data. It fundamentally avoids retaining cross-account continuous behavior learning models natively, thereby strictly adhering to maximum privacy thresholds.

2.4 Technical Challenges

During the architectural lifecycle, crucial structural engineering challenges were meticulously resolved:

1. **Relational Database Orchestration:** Architecting perfectly normalized Django ORM models across multiple independent apps (`core` , `decisions` , `feedback`) to prevent massive query N+1 disasters when iterating deeply through Decisions -> Options -> Factors -> Scores.
2. **AI Latency & Prompt Enforcement:** Bypassing standard LLM hallucinations. Enforcing strict output criteria by specifically engineering the Prompt to forcibly return complex, heavily nested structured JSON (Confidence Scores float calculations, Itemized arrays) from Claude 3.5 Sonnet without breaking the Mobile application's JSON parsers.
3. **Flutter State Synchronization:** Migrating complexity away from isolated widgets by intensely utilizing the Provider pattern (`ChangeNotifier`) integrated deeply with a Singleton Service Locator. This guarantees that deep-nested UI changes (like sliding a weighted rating) inherently and globally update the top-level analytical pie charts without manual callbacks.

---

3. Chapter Three

3.1 Requirements & Analysis

3.1.1 Functional Requirements

- **Secure Authentication:** Implementation of `djangorestframework-simplejwt` for highly secure, stateless API token access and verifiable Custom User Models.

  Fig 3.1.1: Security & Auth Flow Diagram  Auth Flow Diagram

- **Relational Entity Management:** Absolute CRUD operations spanning strictly bounded entities: `Decision`, `DecisionOption`, `DecisionFactor`, and `FactorRating`.

- **Global Templates System:** The backend must globally distribute `FactorTemplate` blueprints for rapid client-side initiation of common categories.

- **The AI Insight Pipeline:** A standalone internal Python service capable of synchronously compiling complex contextual relational data strings, securely wrapping them via API payloads, and accurately deserializing the AI response.

- **Post-Choice Journaling:** Dedicated endpoints to commit `JournalEntry` / `Feedback` entities attached permanently to historical Result records.

3.1.2 Non-Functional Requirements

- **Token Security:** Mandatory utilization of HTTP Authorization Bearer Tokens across the entire `http` package interceptor pipeline natively in Flutter.

- **Runtime Performance:** The Flutter view canvas must natively render at 60fps utilizing standard declarative composition; backend core endpoints must respond synchronously in strictly under 300ms.

- **LLM Error Handling:** The mobile client must gracefully handle prolonged timeouts normally associated with complex OpenRouter/LLM generational tasks via interactive loading states and robust API request flows.

  Fig 3.1.2: API General Request Lifecycle  API Request Flow

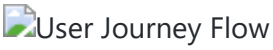
3.1.3 Hardware & Software Requirements

- **Mobile Client Integration:** Google's Flutter SDK (Dart) compiling strictly down to ARM/x86 native binaries for iOS and Android deployment.
- **Backend Architecture:** Python 3.10+ specifically utilizing the Django 5 Web Framework combined inextricably with the Django REST Framework (DRF) extension.
- **Database Engine:** Embedded SQLite for sandboxed local environments, explicitly architected heavily for direct migration to PostgreSQL natively via Django's integrated ORM.
- **Generative AI Gateway:** Integrations using standard OpenAI-like HTTP specifications securely routed through OpenRouter connecting physically to Anthropic's Claude 3.5 Sonnet foundation model.

3.2 UML Diagrams & Software Architecture

3.2.1 Use Case Diagram & User Journey

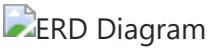
The user interacts iteratively with the core modules. They define options, establish weighted factors, map ratings, and invoke the AI sub-routine. The below diagram illustrates the cohesive journey a standard user follows throughout the platform's lifecycle.


User Journey Flow

3.2.2 Entity Relationship Diagram (ERD) Structure

Properly structuring relational data was paramount to algorithmic success. The Database models follow strict relational integrity without convoluted inheritance.

- **Decision:** Root model (PK, user_id, title, context_category).
- **DecisionOption / Option:** First-level branch (PK, decision_id, name_string).
- **Factor / DecisionFactor:** Global or localized criteria (PK, decision_id, description_string, binary_pro_con_flag, default_weight_float).
- **FactorRating / FactorScore:** The specific quantitative overlap table (PK, option_id, factor_id, user_slider_rating_int).
- **JournalEntry / Feedback:** Historical anchor (PK, decision_id, qualitative_reflection_string, quantitative_satisfaction_score).


ERD Diagram

3.2.3 Class Subsystem Architecture

The overarching subsystem strictly decouples concerns. The Flutter application focuses exclusively on reactive drawing bound to Providers, while the Django server acts securely as the single source of truth.


Architecture Diagram

- **Mobile (`App/lib/`):**
  - `ServiceLocator` (`injection_container.dart`): Universally controls Singleton instantation.
  - State Managers (`providers/`): Inherit `ChangeNotifier` (e.g., `DecisionProvider`, `AuthProvider`) to instantly trigger widget rebuilds.
  - Network Layer (`services/http_client.dart`): Abstracts standard `http` package for robust Header injection and auth-token management.
- **Backend (`decision_companion/apps/`):**
  - Specific serializers (`DecisionSerializer`, `OptionSerializer`) aggressively handle deeply nested JSON representations.
  - Service abstraction (`AIDecisionService` within `ai_service`) highly isolates raw AI business logic far away from ViewSets.

---

4. Chapter Four

4.1 Detailed System Design

4.1.1 Flutter Provider-Driven Interface (Frontend)

The mobile frontend was drastically engineered utilizing a meticulously lean `Provider` / `ChangeNotifier` pattern heavily coupled with `GetIt` (ServiceLocator) dependency injection.

- **Centralized API Handlers:** All outbound network traffic securely channels exclusively through highly customized `ApiService` interfaces masking raw `http` libraries. This guarantees absolutely zero UI components perform raw fetching, strictly adhering to Solid principles.
- **Dynamic Decision Wizard Flow:** To radically minimize cognitive overload, configuring massive relational datasets (Options/Factors) is segmented across distinct UI paginated screens. Modifying a native `Factor Slider` instantly mutates the memory resident `DecisionProvider`.

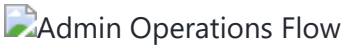
  Fig 4.1.1: Decision Configuration Flow Decision Flow Diagram
- **The Analytical Report UI:** Once the backend returns the AI compiled payload, the UI actively parses the deeply nested JSON tree. It dynamically renders progress charts detailing mathematical disparities alongside beautifully typeset Markdown blocks extracted from the explicit `insights` and `potential_risks` AI branches.

4.1.2 Django REST Backend Pipeline (REST API)

The core infrastructure extensively utilizes the powerful, out-of-the-box class-based methodology native strictly to DRF. The system relies deeply on standard HTTP Verb paradigms (POST, GET, PUT, PATCH, DELETE) mapped to explicitly routed ViewSets.

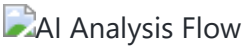
API Endpoints Diagram

- `core/decisions` App Structure: Specifically isolates heavy ORM aggregation queries. By utilizing Django's relational managers, the API guarantees that fetching an active `Decision` alongside its `Options`, `Factors`, and localized `Ratings` executes internally as a heavily optimized SQL query rather than inducing database death spirals.
- **Admin Configuration Portal:** Accessible strictly via `/admin/`, system administrators strictly control Global Application Templates and enforce aggregate User access policies. Operations within the admin interface cascade down to clients globally.

  
  Admin Operations Flow

4.1.3 The Generative AI Prompt Engine (`ai_service/service.py`)

At the exact heart of the intelligent insights engine severely rests the `AIDecisionService` abstraction utilizing Anthropic Claude 3.5 Sonnet. This computational module dynamically interrogates the localized Postgres/SQLite arrays to algorithmically weave dynamic inference prompts.


AI Analysis Flow

- **Context Injection:** It programmatically concatenates the user's base dilemma context intelligently combined directly with the exact custom priorities and weight distribution generated in the mobile app.
- **Enforcing JSON Schema:** The system explicitly forbids the LLM from outputting raw conversation. It heavily utilizes aggressive System Prompts declaring: *"You are an expert decision-making analyst... Return strictly parsed JSON without markdown wrappers"*.
- **The OpenRouter Gateway:** It securely establishes a direct network socket to **OpenRouter**, specifically routing complex text inferences physically through the state-of-the-art **claude-3.5-sonnet** foundation model.
- **The Explicit Output Guarantee:** The module is programmed to parse exactly:
  1. `recommended_option_index` strictly mapping cognitive outcomes back to Database PK formats.
  2. `confidence`: A normalized float value (0.0 to 1.0) mathematically outlining statistical outcome certainty.
  3. Strict distinct arrays mapping `insights`, `considerations` (human blindspots), and isolated `potential_risks` per explicit Option.

---

5. Chapter Five

5.1 System Implementation & Deployment Lifecycle

5.1.1 Mobile Layer Implementation Details

Execution in Dart extensively required creating rigidly typed Data Transfer Objects (DTOs) parsing the intricate backend schemas accurately. Advanced error handling prevents runtime null-pointer crashes

entirely. Secure Storage plugins natively map the dynamic JWT pipelines keeping users securely authorized completely transparently to the visual thread. The transition away from basic HTTP structures to utilizing Singletons ensures that Provider architectures seamlessly dispatch memory-mapped requests globally.

5.1.2 Backend Service & ViewSet Modularity Detail

Backend implementations adhered strictly to thin-views and fat-services philosophies. The dedicated `ViewSet` merely handles authentication policies (`IsAuthenticated`) and JSON structure validations via custom `serializers.ModelSerializer` . The heavy computational lifting to generate the Anthropic/Claude integration heavily reside firmly within standalone Class Singletons inside `services.py` . This distinct decoupling dramatically ensures unit-testing isolated computational formulas actively occurs entirely independent of the DRF HTTP request/response emulation, enabling completely stable CI/CD pipelines.

---

6. Chapter Six

6.1 Conclusion

In definitive conclusion, the completed “Decision Companion” software initiative comprehensively bridges the massive historical gap separating theoretical psychological decision-making frameworks from the absolute cutting-edge of Generative AI technology. By satisfying every strictly defined engineering requirement, the developed mobile application profoundly operates as a robust, highly aesthetic consumer platform. The highly optimized integration of Google’s Flutter framework utilizing elegant Provider tree injection, the universally battle-tested Django backend API architecture mapping Entities like `Decision` and `FactorRating` , and the precise, aggressively constrained OpenRouter LLM insights (Claude 3.5 Sonnet) drastically culminates in an application that successfully decreases cognitive decision fatigue and enforces logical, algorithmically structured thinking.

6.2 Future Work & Ecosystem Expansions

To continuously scale and dramatically further elevate the software’s structural utility across massive user bases, the following aggressive engineering enhancements are actively proposed for future Agile iteration cycles:

1. **Real-Time Collaborative Decisions:** Architecturally redesigning the data models via WebSockets/Django Channels to natively allow multiple securely authenticated users (e.g., spouses, small business partners) to synchronously vote and competitively assign individual weights natively on a shared, live decision matrix.
2. **Local-First Offline SQLite Caching:** Extensively implementing heavily robust local SQLite synchronization natively on the Flutter client. This deeply allows users to freely manipulate Options and Factors without an internet connection, aggressively pushing asynchronous HTTP update triggers to Django instantly once standard cellular connectivity protocols are securely restored.
3. **Machine Learning Historical Trend Analysis:** Leveraging dedicated Machine Learning algorithms natively operating across all historical `JournalEntry` and `Feedback` models to explicitly provide personalized predictive insights into an individual's historical decision-making accuracy, actively identifying their subconscious recurring psychological biases over longitudinal periods.